CHAPTER 30

Batch Processing, Prompt Caching and Cost Controls

In this chapter

Introduction: The Fifty Per Cent Discount That Cost Sixty-Two Per Cent More . . . 3
30.1 Core Concepts: Three Levers and One Unit . . . 4
30.2 The Message Batches API . . . 6
30.3 What a Batch Refuses, and Why . . . 7
30.4 Extended Output: 300,000 Tokens, Batch Only . . . 8
30.5 Prompt Caching: Prefix, Hash, Breakpoint . . . 8
30.6 The Twenty-Block Lookback Window . . . 9
30.7 Automatic Caching . . . 9
30.8 The Price of a TTL . . . 10
30.9 Mixing TTLs: The A, B and C Positions . . . 12
30.10 What Invalidates the Cache . . . 12
30.11 Cache Diagnostics (Beta) . . . 13
30.12 Pre-warming with max_tokens: 0 . . . 14
30.13 Cost Controls . . . 15
30.14 Visual Analogies . . . 15
30.15 Architecture: Where the Three Levers Sit . . . 16
30.16 Figures . . . 17
30.17 Examples . . . 18
30.18 Code Walkthrough: The Algorithm, Not the Advice . . . 19
30.19 Mini Project: A Batch-versus-Online Decision Model, a Cache Planner and a Preflight . . . 20
30.20 Hands-on Labs . . . 24
30.21 Lab Solutions . . . 24
30.22 Best Practices . . . 25
30.23 Common Mistakes . . . 25
30.24 Performance Considerations . . . 27
30.25 Security Considerations . . . 27
30.26 Real Production Examples . . . 28
30.27 Certification Notes . . . 29
30.28 Twenty Practice Questions . . . 29

30.29 Answers 30
30.30 Ten Practical Exercises 31
30.31 Five Challenge Labs 32
30.32 Ten Interview Questions 32
30.33 Summary 32
30.34 Glossary 34
30.35 Further Reading 35

Part 5 of 13 · Chapter 30 of 64. Next: Chapter 31 — How Tool Use Works (Part 6 opens).

NOTE

Learning Objectives

By the end of this chapter you will be able to:

1. Decide between the synchronous Messages API and the Message Batches API on evidence — cost, rate-limit draw, payload size and latency — rather than on the headline 50 per cent discount.
2. Enumerate everything the batch path refuses, reproduce each rejection offline, and say why each rule exists rather than memorising a list.
3. Explain prompt caching as a **cumulative prefix hash with writes only at breakpoints**, and predict from that mechanism alone which edits cost you the cache.
4. Apply the documented **20-block lookback window** and place a second breakpoint before a growing conversation outruns it.
5. Price a cache configuration in base-input units, and compute the break-even hit rate above which caching is cheaper than not caching at all.
6. Choose between the 5-minute and 1-hour TTLs from the **interarrival distribution of requests sharing a prefix**, not from the length of the prompt.
7. Read `cache_creation_input_tokens`, `cache_read_input_tokens` and `input_tokens` correctly, and connect each to money, to ITPM and to the context window.
8. Assemble the cost controls — workspace spend caps, the Usage and Cost API, `inference_geo`, batching and caching — into a plan whose numbers you can defend.

NOTE

Prerequisites

Chapter 11 for the two-term latency model, the rate card and the first appearance of a misplaced cache prefix. **Chapter 16** for the request body and statelessness as a quadratic cost. **Chapter 19** for the three buckets, cache-aware ITPM, tiers and spend caps, and the Batch API's separate RPM and queue. **Chapter 21** for the hashed `tools` → `system` → `messages` prefix and the mutating prefix that cost exactly 1.25×. **Chapter 26** for workspace scoping, beta headers and the 413. **Chapter 27** for `max_tokens` as a hard ceiling. **Chapter 28** for the `usage` object and `stop_reason`. **Chapter 29** for retries, timeouts and the fact that a client-side timeout does not cancel the work.

Deferred by name: **caching tool definitions in an agent loop** is revisited in **Chapter 35**; **context compaction and summarisation**, which change what is cacheable at all, are **Chapter 54**; **scaling, caching and cost optimisation as a production discipline** is **Chapter 60**; **compliance, governance and data residency** is **Chapter 61**.

Introduction: The Fifty Per Cent Discount That Cost Sixty-Two Per Cent More

A team runs a nightly triage job: 400,000 support tickets, each classified against a taxonomy. The prompt is a 20,000-token taxonomy with twenty worked examples, followed by roughly 500 tokens of ticket, producing about 800 tokens of structured verdict. On Claude Opus 5, running online with prompt caching working properly, it costs **$13,920** a night.

Someone reads that the Message Batches API charges **50 per cent of standard prices** and moves the job across. The bill goes to **$22,600**.

Nothing was mis-configured. The discount is real and it applied to every token. What changed is the thing nobody priced: **cache hit rate**. Online, the 400,000 requests arrive back to back against one shared prefix, and essentially every one of them reads it — the measured configuration in §30.19 runs at 0.98. In a batch, the documentation is explicit that *"because batch requests are processed asynchronously and concurrently, cache hits are provided on a best-effort basis. Users typically experience cache hit rates ranging from 30% to 98%."* At the bottom of that band the batch pays a 1.25× **cache write** on 70 per cent of 400,000 requests. Half of a large number is larger than a tenth of the same number.

The crossover, computed exactly in §30.19, is a batch cache hit rate of **67.74 per cent**. Above it, batching wins. Below it, batching loses to the online path you already had — and the whole documented 30-to-98 band straddles the line, so the answer for your workload is genuinely not knowable from the price list.

That is this chapter's shape. Three levers — **batching**, **caching** and **cost controls** — each of which is usually described as a saving, and each of which has a regime where it costs you money. Chapter 21 already met one: a `cache_control` breakpoint on a block that changes every request costs **exactly 1.25 times** not caching at all. This chapter finds the general rule behind that number, and two more like it.

The general rule is one line of algebra. A cached prefix costs `(1 − h) × w + h × 0.1` per base-input unit, where `h` is the hit rate and `w` is the write multiplier. Not caching costs `1.0`. Setting them equal gives the break-even hit rate:

```
h* = (w − 1) / (w − 0.1)
```

For the 5-minute cache, `w = 1.25` and `h* = 21.74%`. For the 1-hour cache, `w = 2.0` and `h* = 52.63%`. Below those hit rates, the correct configuration is **no** `cache_control` **at all** — and the API will neither warn you nor error.

There is a second, less comfortable result waiting behind it. Convert those hit rates into arrival rates using a memoryless arrival process, and the 5-minute cache pays only above **2.94 requests per hour on the same prefix**, while the 1-hour cache pays above **0.75**. Because the 1-hour cache is cheaper than the 5-minute cache at every rate above **0.55 per hour**, and neither is worth buying below 0.75, **the 5-minute default is strictly optimal at no arrival rate whatsoever** under that model. §30.19 publishes that as a negative result, because it contradicts documented guidance, and then publishes the distribution class that overturns it: bursty session traffic, where the 5-minute cache is **27.78 per cent cheaper** than the 1-hour one. Both results are in the test suite.

Finally, a warning about the batch payload. Every request in a batch carries its own complete copy of the prompt — batches are as stateless as the Messages API, and Chapter 16's quadratic statelessness applies unchanged. A 20,500-token prompt at four bytes per token is 82 kB per request, so **256 MB holds 3,273 requests, not 100,000**. The nightly job needs **123 batches**, not four. The limit that binds is almost never the one people quote.

30.1 Core Concepts: Three Levers and One Unit

Define the vocabulary before using it.

DEFINITION

Message Batches API

An asynchronous endpoint that accepts a list of Messages requests, processes each independently, and makes results available when all requests have finished **or after 24 hours, whichever comes first**. Charged at **50 per cent** of standard prices on both input and output.

DEFINITION

custom_id

A caller-supplied identifier on each request in a batch, matching `^[a-zA-Z0-9_-]{1,64}$` and unique within the batch. Results *"can be returned in any order"*, so `custom_id` is the only correct way to match a result to its request.

DEFINITION

processing_status

The batch's lifecycle field: `in_progress` → `ended`, with `canceling` in between if you cancel. A cancelled batch still ends as `ended` and *"may contain partial results"*.

DEFINITION

Prompt cache

A key-value store, held in memory, mapping a **cumulative hash of a prompt prefix** to the model's internal representation of that prefix. A hit skips prefill for those tokens. It changes cost and latency; it does not change the answer.

DEFINITION

Cache breakpoint

A `cache_control: {"type": "ephemeral"}` marker. It is where a **write** happens, and nowhere else. Up to **four** per request.

DEFINITION

TTL

The lifetime of a cache entry: **5 minutes** (default, write costs 1.25× base input) or **1 hour** (`"ttl": "1h"`, write costs 2× base input). Reads cost **0.1×** and refresh the entry at no charge.

DEFINITION

Lookback window

The **20 positions** the system searches backwards from a breakpoint for a prefix hash a previous request already wrote. Twenty is a count of blocks, not tokens.

DEFINITION

Cache pre-warming

A request with `max_tokens: 0`, which reads the prompt, writes the cache at your breakpoints and returns immediately with an empty `content` array and `stop_reason: "max_tokens"`. It targets **time-to-first-token**, not cost.

The unit that ties them together is the **base-input unit**: one token priced at the model's standard input rate. Every input token in this chapter costs `0.1`, `1.0`, `1.25` or `2.0` of those, and the Batch discount halves whichever one applies. Reasoning in these units rather than in dollars keeps the arithmetic model-independent — Chapter 11 established that output is exactly 5× input on every current model, so a single ratio carries across the whole family.

30.2 The Message Batches API

A batch is a list. Each entry is a `custom_id` and a `params` object containing an otherwise ordinary Messages request. The documented shape, verified 2 August 2026:

```
message_batch = client.messages.batches.create(
    requests=[
        Request(
            custom_id="ticket-88401",
            params=MessageCreateParamsNonStreaming(
                model="claude-opus-5",
                max_tokens=1024,
                messages=[{"role": "user", "content": "Classify..."}],
            ),
        ),
    ]
)
```

The response carries `processing_status: "in_progress"`, a `request_counts` object with five counters (`processing`, `succeeded`, `errored`, `canceled`, `expired`), `created_at`, `expires_at` exactly 24 hours later, and `results_url: null`.

Four facts about that object decide how you write the client.

Validation is asynchronous. *"Validation of the* `params` *object for each message request is performed asynchronously, and validation errors are returned when processing of the entire batch has ended."* Submit 100,000 requests with a malformed field and you learn about it up to 24 hours later. The documented mitigation is to dry-run one request shape against the synchronous Messages API first; the mitigation this chapter builds is a local preflight that reproduces the documented rejections without a key at all.

Results are `.jsonl`**, unordered, and streamed.** Each line is one result with a `custom_id` and a `result` whose `type` is one of four values:

Result type	Meaning	Billed?
`succeeded`	Includes the `message`	Yes
`errored`	Invalid request or internal server error	**No**

Result type	Meaning	Billed?
`canceled`	You cancelled before it was sent to the model	**No**
`expired`	The 24-hour window closed first	**No**

That billing column matters for the failure budget. Chapter 29 measured retries that cost money because a client-side timeout does not cancel the generation; on the batch path, an expiry is free. The cost of a batch that expires is the **latency**, not the invoice.

The limits are two, and either can bind. *"A Message Batch is limited to either 100,000 Message requests or 256 MB in size, whichever is reached first."* §30.19 shows that for any prompt with a substantial shared prefix, the byte limit binds first by a wide margin.

Retention is 29 days from `created_at`. Not from `ended_at` — the troubleshooting section says so explicitly. A batch created on the 1st and finished on the 2nd loses its results on the 30th, not the 31st.

Batches are scoped to a **workspace** (Chapter 26), can be listed with the standard `after_id`/`before_id` pagination, can be cancelled, and can be deleted with `DELETE /v1/messages/batches/{batch_id}` — though an in-progress batch must be cancelled before it can be deleted.

30.3 What a Batch Refuses, and Why

Six parameters are rejected on the batch path. Each rejection has a stated reason, and the reasons are more instructive than the list.

Parameter	Documented reason
`stream: true`	*"Batch results come back as a single file, not a stream."*
`speed` (fast mode)	Fast mode tunes **synchronous** latency
`store` / `previous_thread_event_id`	*"Threads are stateful; batch requests are not."*
`cache_hint` / `context_hint`	Routing hints apply to synchronous scheduling only
`max_tokens: 0`	Cache pre-warming; see §30.12
`research_preview_2026_02: "active"`	Not available on the batch path

Add one more from Chapter 25, verified 1 August 2026: **server-side fallback is rejected on the Batches API**, and a batch result's `fallback_credit_token` cannot be redeemed.

The through-line is that every rejected parameter is about *when* a response arrives or *what connection carries it*. A batch has no connection and no schedule you control, so every knob that tunes a connection or a schedule is meaningless. This is a much better thing to remember than the table.

What **is** supported is nearly everything else: vision, tool use including all server tools, system messages, multi-turn conversations, extended thinking, prompt caching, and *"most beta features"*. All active models are supported.

One behaviour changes rather than disappearing. Server tools run in a batch, and *"because there is no open connection to maintain, the batch loop runs **more iterations per turn** than a synchronous request before it returns* `stop_reason: "pause_turn"`." Chapter 28 established that `pause_turn` is the server's loop and never yours, with a documented default of 10 iterations per request; on the batch path that default is higher and is not published. A batch result carrying `pause_turn` is an unfinished turn you must continue — in a follow-up batch or synchronously. Additionally, `web_search` is throttled per organisation inside the batch worker so that concurrency cannot exhaust the organisation's search limit; throttled calls are retried automatically, which is another reason a large search-heavy batch can take longer.

30.4 Extended Output: 300,000 Tokens, Batch Only

The beta header `output-300k-2026-03-24` raises the `max_tokens` cap from 128,000 to **300,000**, on **Claude Opus 5, Opus 4.8, Opus 4.7, Opus 4.6, Sonnet 5 and Sonnet 4.6**. Claude Fable 5 and Claude Haiku 4.5 are not on the list. Claude Mythos 5 appears on no list at all, which by this book's rule from Chapter 14 makes it **UNKNOWN**, and §30.18 shows the preflight raising rather than guessing.

Three constraints travel with it. It is *"available on the Message Batches API only, not the synchronous Messages API."* It is supported on the Claude API and Claude Platform on AWS, and *"not currently available on Amazon Bedrock, Google Cloud, or Microsoft Foundry."* And *"a single 300k-token generation can take over an hour to complete"*, against a 24-hour batch window — so a batch of many such requests needs planning, not optimism.

Chapter 27's rule is unrepealed: `max_tokens` remains a hard ceiling on **thinking plus text**. Three hundred thousand tokens is not three hundred thousand tokens of prose if the model is thinking.

30.5 Prompt Caching: Prefix, Hash, Breakpoint

Chapter 21 established the ordering: the cached prefix is hashed `tools` → `system` → `messages`, so a change to `tools` invalidates everything and a change to the top-level `system` field invalidates everything after it. This chapter can now state the mechanism precisely, because the documentation does. Three principles, quoted and then unpacked:

One. Writes happen only at your breakpoint. *"Marking a block with* `cache_control` *writes exactly one cache entry: a hash of the prefix ending at that block. The system does not write entries for any earlier position."*

Two. Reads look backward for entries that prior requests wrote. *"On each request the system computes the prefix hash at your breakpoint and checks for a matching cache entry. If none exists, it walks backward one block at a time... It is looking for prior writes, not for stable content."*

Three. The lookback window is 20 blocks. *"The system checks at most 20 positions per breakpoint, counting the breakpoint itself as the first."*

Principle two is the one that costs people money, because the intuitive model is wrong in a specific way. The intuitive model is *"the system caches whatever part of my prompt is stable."* It does not. It caches **whatever prefix a previous request explicitly asked it to write**, and then looks for that. Stability is necessary and not sufficient; a breakpoint is also required, and it has to have been there on an earlier

request.

That gives the failure the documentation names and Chapter 21 measured. Put five stable system blocks in front of one block containing a timestamp and the user's message, and set `cache_control` on the last block:

- Request 1 hashes blocks 1–6, including the timestamp, and writes an entry.
- Request 2 has a different timestamp, so the hash at block 6 differs. The lookback walks back through blocks 5, 4, 3, 2, 1 — and **the system never wrote an entry at any of those positions**, because writes happen only at breakpoints. No hit.

Every request pays a 1.25× write and never a read. $30.19 measures ten such requests at **exactly 1.25 times** the cost of no caching at all, reproducing Chapter 21's figure from the mechanism rather than from a coincidence. Moving the breakpoint one block earlier, to the last block that is identical across requests, takes the same ten requests to **0.2173** times the uncached cost — a **5.75-fold** swing from moving one marker.

> WARNING
>
> **The rule to memorise**
>
> Place `cache_control` on **the last block whose prefix is identical across the requests you want to share a cache**. Not the last block. Not the biggest block. The last **identical** one.

30.6 The Twenty-Block Lookback Window

The window is counted in blocks, and blocks are not tokens. A single `tool_result` is one block whether it holds 12 tokens or 12,000.

The documentation's own worked example: turn 1 has 10 blocks with a breakpoint on block 10, and writes an entry there. Turn 2 has 15 blocks with a breakpoint on block 15; block 15 has no entry, so the system walks back to block 10, finds the turn-1 entry, processes only blocks 11–15 fresh and writes a new entry at 15. Turn 3 has 35 blocks; the system checks blocks 35 down to 16, finds nothing, and the turn-2 entry at block 15 is **one position outside the window**. No hit, and a full re-prefill.

The general condition falls out of that immediately. If each turn's breakpoint sits at the final block, a hit requires the previous turn's breakpoint to be within 20 positions, which means each turn may add at most **19 blocks**. $30.19 sweeps it and finds a cliff, not a slope: at 19 added blocks per turn the hit rate is 87.5 per cent (everything but the first turn); at 20 it is **0.0 per cent**. Nothing degrades gracefully.

This is an agent-loop problem more than a chat problem. A conversational turn adds two blocks. A tool-using turn that fires twelve parallel tools and returns twelve results adds twenty-five, and Chapter 28's rule that the continuation must be *"nothing except* `tool_result` *blocks"* means you cannot merge them to duck under the limit. The documented fix is a second breakpoint placed closer to the growing end **from the start**, so that a write accumulates there before you need it: $30.19 restores 7 of 8 turns to a hit by adding one.

30.7 Automatic Caching

Automatic caching is a single top-level `cache_control` field on the request body. *"The system automatically applies the cache breakpoint to the last cacheable block and moves it forward as conversations grow."*

It is the right default for a growing conversation, because it solves exactly the problem §30.6 describes: the breakpoint tracks the end of the message list, so the previous turn's write is always the most recent position and the lookback finds it — provided each turn adds fewer than 20 blocks.

It is the wrong default for a prompt with a varying suffix, and the documentation says so in as many words: *"Automatic caching hits the same trap: it places the breakpoint on the last cacheable block, which in this structure is the one that changes every request, so use an explicit breakpoint on block 5 instead."* Automation places the marker correctly for one prompt shape and incorrectly for the other.

The mechanics to know:

- It consumes **one of the four** breakpoint slots, and combines with explicit breakpoints.
- If the last block already carries an explicit `cache_control` **with the same TTL**, automatic caching is a no-op.
- If it carries one with a **different** TTL, the API returns a **400**.
- If four explicit breakpoints already exist, the API returns a **400** — no slot left.
- If the last block is not an eligible target, *"the system silently walks backwards to find the nearest eligible block. If none is found, caching is skipped."*
- TTL is set by `{"cache_control": {"type": "ephemeral", "ttl": "1h"}}`.
- Available on the Claude API, Claude Platform on AWS, Google Cloud and Microsoft Foundry. **Amazon Bedrock does not support automatic caching.**

Everything else — pricing, minimum lengths, the ordering requirement and the 20-block window — is identical to explicit breakpoints.

30.8 The Price of a TTL

The multipliers are constant ratios on the base input price, unchanged since Chapter 11: **1.25× a 5-minute write, 2× a 1-hour write, 0.1× a read**, and *"these multipliers stack with other pricing modifiers such as the Batch API discount and data residency."* Both directions of that stacking matter: a 1-hour write inside a batch costs `2 × 0.5 = 1.0` base units, exactly the price of not caching at all outside a batch.

The break-even follows from one equation, given in the introduction and derived in full in §30.19. Expected cost per base-input unit is `(1 − h) × w + h × 0.1`; not caching is `1.0`; equality at `h* = (w − 1) / (w − 0.1)`.

TTL	Write multiplier	Break-even hit rate	Break-even reuse
5 minutes	1.25×	**21.74%**	2nd request wins
1 hour	2.00×	**52.63%**	3rd request wins

The reuse column is the same statement counted rather than averaged: one 5-minute write plus one read is 1.35 against 2.0 for two uncached requests, so caching wins on the **second** request; a 1-hour write plus

one read is 2.1 against 2.0, so it does not win until the **third**.

Hit rate is not something you choose. It is a consequence of how often the same prefix comes back. Model the gaps between requests sharing a prefix as a memoryless process of rate per hour, and the probability the entry is still alive is `1 − exp(−λT)`. Inverting at `h*` gives an arrival rate:

TTL	Break-even arrival rate on one prefix
5 minutes	**2.9415 requests/hour**
1 hour	**0.7472 requests/hour**

Below roughly three requests an hour on the same prefix, a 5-minute `cache_control` marker is **a way of paying 25 per cent more for your input tokens**. Nothing errors. `cache_creation_input_tokens` is populated on every request and `cache_read_input_tokens` is zero on every request, which looks like the feature working.

§30.19 pushes this to its uncomfortable conclusion and publishes the result as a negative one, because it disagrees with documented guidance. It also publishes the traffic shape that rescues the 5-minute cache — and that shape, sessions separated by long silences, is what most human-facing products actually look like.

> WARNING
>
> **The diagnostic**
>
> Two zeros mean two different things. `cache_creation_input_tokens` **and** `cache_read_input_tokens` both zero means nothing was cached — most often the prompt was below the model's minimum, and *"no error is returned"*. A large `cache_creation_input_tokens` with a persistent zero `cache_read_input_tokens` means the write is happening and the read never will: your breakpoint is on a block that changes.

The minimum cacheable prompt length deserves its own table, because it is **not monotonic in model recency** and the exceptions are large:

Model	Minimum cacheable tokens
Claude Opus 5, Fable 5, Mythos 5	**512**
Claude Opus 4.8, Sonnet 5, Sonnet 4.6, Sonnet 4.5	1,024
Claude Mythos Preview, Opus 4.7	2,048
Claude Opus 4.6, Opus 4.5	4,096
Claude Haiku 4.5	**4,096**

Haiku 4.5 needs **eight times** Opus 5's minimum. A 2,000-token system prompt that caches beautifully on Opus 5 caches not at all on Haiku 4.5, silently, and Chapter 15's efficiency-first strategy — start on Haiku, upgrade for capability — walks straight into it. Note also a small discrepancy on the page itself, recorded here rather than smoothed over: the pricing table describes Claude Haiku 3.5 as *"retired, except on Bedrock and Google Cloud"* while the cache-limitations list on the same page says *"retired, except on*

Google Cloud". Neither reading affects a current build; the practice of writing the disagreement down is the point.

30.9 Mixing TTLs: The A, B and C Positions

You may use both TTLs in one request, with one constraint: *"Cache entries with longer TTL must appear before shorter TTLs."* A 1-hour block cannot follow a 5-minute one.

Billing is then defined by three token positions:

- **A** — the token count at the **highest cache hit** (0 if there were none).
- **B** — the token count at the highest **1-hour** `cache_control` block after A (equal to A if none).
- **C** — the token count at the **last** `cache_control` block.

And the bill is: **read** tokens for A, **1-hour write** tokens for (B − A), **5-minute write** tokens for (C − B), and ordinary input for everything after C.

The design this licenses is a layered prefix. Tool definitions and a policy document change weekly, so they go behind a 1-hour breakpoint; today's caseload changes daily but is re-read constantly, so it goes behind a 5-minute breakpoint; the question is last and is never cached. §30.19 runs exactly that shape and shows the first request billing `write_1h = 14,000`, `write_5m = 3,000`, `base = 400`, and the second request billing `read = 17,000` with both write columns at zero.

The response reports the split under `usage.cache_creation`:

```
"cache_creation": {
  "ephemeral_5m_input_tokens": 148,
  "ephemeral_1h_input_tokens": 100
}
```

`cache_creation_input_tokens` equals the sum of that object. Chapter 28's accountant should read the object, not the scalar, the moment two TTLs are in play.

One surprise belongs here. If you use server tools with caching enabled, *"the API automatically places a cache breakpoint on the server tool result before running the next iteration of the agentic loop"*, and *"this automatic breakpoint always uses the default 5-minute TTL, independent of any TTL you set."* So a request in which every `cache_control` you wrote says `"1h"` can still report `ephemeral_5m_input_tokens` writes you never asked for. That behaviour only applies when the request already has at least one `cache_control` marker. Whether that automatic breakpoint consumes one of your four slots is documented neither way, and this book records it as **UNKNOWN**.

30.10 What Invalidates the Cache

The hierarchy is `tools` → `system` → `messages`; a change at one level invalidates that level and everything after it. The documented table, with meaning invalidated:

Change	tools	system	messages
Tool definitions (names, descriptions, parameters)			
Web search or citations toggled			
`speed` switched between fast and standard			
`tool_choice` changed			
`disable_parallel_tool_use` changed			
Images added or removed anywhere			
Thinking configuration changed	model-specific	model-specific	
`output_config.effort` changed	model-specific	model-specific	

Chapter 18's running thread survives intact: **the configuration is part of the prompt, so it is part of the cache prefix**. Effort and thinking are rendered into the prompt, so changing them always invalidates the message cache — and *"setting effort explicitly to the model's default is equivalent to omitting it and does not invalidate."*

Two further items are worth pinning down. Thinking blocks *"cannot be cached directly with* `cache_control`*"* but **are** cached alongside other content in previous assistant turns, and *"when cached this way, they DO count as input tokens when read from cache"* — Chapter 18 measured 76.6 per cent of input tokens at 20 turns being carried thinking, and caching changes their price, not their presence. And `defer_loading` with the tool search tool is the one way to add tools mid-conversation without invalidating anything: a discovered tool arrives as a `tool_reference` block **in the message history**, leaving the `tools` prefix untouched.

Finally, a correction to record. Chapter 21 reported that mid-conversation system messages were documented on **Claude Opus 4.8 only**, taking the narrower of two readings and publishing the discrepancy. As of 2 August 2026 the prompt-caching page states that on **Claude Fable 5, Claude Mythos 5, Claude Opus 4.8 and Claude Opus 5** you may append a `{"role": "system"}` message *"without invalidating the system or message caches"*, and adds explicitly that *"this feature is not available on Claude Sonnet 5."* The two pages still disagree on breadth. The narrow reading remains the safe one to build against; the caching argument for the feature is unaffected either way, and it is a good one — appending to `messages` leaves the hashed prefix untouched, where editing the top-level `system` field invalidates everything after it.

30.11 Cache Diagnostics (Beta)

When a cache misses, `cache_read_input_tokens` drops to zero and tells you nothing about why. Cache diagnostics closes that gap: send the beta header `cache-diagnosis-2026-04-07` and pass `diagnostics: {"previous_message_id": <id of the previous response>}`, and the API compares fingerprints of the two requests and reports **the first point of divergence**.

Six `cache_miss_reason` types, as a discriminated union on `type`:

Type	Cause	Fix
`model_changed`	A router, A/B test or fallback picked a different model; the cache is per model	Hold the model constant within a cached conversation
`system_changed`	A timestamp or request ID was interpolated into `system`	Make `system` byte-stable; move dynamic data after the breakpoint
`tools_changed`	Tools added, removed, reordered, or schemas serialised non-deterministically	Fixed order, sorted keys
`messages_changed`	History was edited, truncated or re-serialised rather than appended to	Treat history as append-only; echo content back verbatim
`previous_message_not_found`	No stored fingerprint — *"not evidence that your request changed"*	Send the header on every turn; keep turns close together
`unavailable`	Another prompt-affecting parameter differed, or the divergence was beyond the horizon	Hold `tool_choice`, `thinking`, `context_management`, `output_config` and the beta header set constant

The `*_changed` types carry `cache_missed_input_tokens`, *"derived from byte lengths before tokenization, so treat it as a magnitude indicator rather than a billing number."*

Read it beside `usage`, because the two answer different questions — *"*`diagnostics` *answers 'did my request change?' while* `usage.cache_read_input_tokens` *answers 'did the cache hit?'"*. The row that matters most is `diagnostics: null` **with zero reads**: your requests matched and the entry had simply expired, which is a TTL problem and not a bug — precisely the case §30.8 prices.

Four limits: it is beta and may change; it is **Claude API only**, not Bedrock or Google Cloud; fingerprints expire quickly, so compare closely spaced requests; and the previous request must have come from the same organisation **and workspace**. It never blocks or fails your request.

30.12 Pre-warming with `max_tokens: 0`

A request with `max_tokens: 0` reads the prompt, writes the cache at your breakpoints and returns immediately: `content: []`, `stop_reason: "max_tokens"`, a fully populated `usage`, and **zero output tokens billed**. It replaces the older `max_tokens: 1` trick, which produced a one-token reply you had to discard.

It is a latency instrument, not a cost one. *"Pre-warming targets time-to-first-token."* Chapter 11's two-term latency model is the frame: prefill dominates the first token, decode dominates the rest, and pre-warming moves a large prefill off the user's critical path onto a scheduler's.

Three rules make the difference between a warm cache and a wasted request. Put the breakpoint on **the last block shared with the follow-up request** — not on the placeholder user message, or the entry is keyed to the placeholder and real traffic never hits it. Use **the same thinking configuration and** `output_config.effort` as your real requests, because §30.10's table says those are rendered into the prompt. And therefore use an **explicit** breakpoint, since automatic caching would place it on the placeholder.

Five things are rejected with an `invalid_request_error`, each because it implies output that a zero-token budget cannot produce: `stream: true`, extended thinking (`thinking.type: "enabled"`), structured outputs (`output_config.format`), and `tool_choice` of `{"type": "tool", ...}` or `{"type": "any"}`. And it is rejected inside a batch, for the reason quoted in §30.3.

The TTL still applies: a 5-minute pre-warm must be repeated at least every five minutes to stay warm, which is a scheduled job costing one cache write per interval. Price it against §30.8's break-even before you build it.

30.13 Cost Controls

Caching and batching change what a request costs. Cost controls change what happens when the total gets away from you.

Spend caps stop a month; rate limits throttle a minute. Chapter 19 established the distinction. Caps are **$500 / $1,000 / $200,000 / none** for Start / Build / Scale / Custom, applied at the organisation level, and pause usage until the next calendar month rather than returning a retryable 429. Workspaces carry their own configurable limits. One batch-specific caveat: *"because of high throughput and concurrent processing, batches may go slightly over your Workspace's configured spend limit."* A cap is a brake, not a wall.

The Usage and Cost API is the ground truth. Buckets of `1m`, `1h` and `1d` (maximum 1,440 / 168 / 31), data within about five minutes, polled no more than once a minute. Chapter 26's three gaps still apply: Workbench usage has a null `api_key_id`, the default workspace has a null `workspace_id`, and **Priority Tier costs are absent from the cost endpoint**. Console → Usage → Export gives the same data as CSV by key and model.

`inference_geo: "us"` **costs 1.1× on every token category**, and all `inference_geo` values share one rate-limit pool. It is a compliance control with a price, not an optimisation.

Workspaces isolate the cache. Chapter 26 recorded that prompt caches are isolated **per workspace** on the Claude API, Claude Platform on AWS and Microsoft Foundry, and **per organisation** on Bedrock and Google Cloud. Splitting traffic across workspaces for billing attribution splits the cache too, and a prefix shared by two workspaces is written twice and read half as often. That is a governance decision with a direct line to §30.8's hit rate.

The lever cascade from Chapter 11 now has its full set of rungs, in the order of decreasing certainty: **generate fewer output tokens** (a 5× multiplier), **cache the prefix properly** (up to 10× on the input line, if the arrival rate supports it), **batch what can wait** (a flat 50 per cent), **choose a smaller model or a lower effort** (Chapter 15's (model, effort) pair), and only then argue about geography.

30.14 Visual Analogies

VISUAL ANALOGY

The season ticket

A cache write is buying a season ticket; a read is using it. The 5-minute ticket costs 25 per cent more than a single fare and lasts five minutes; the 1-hour ticket costs a full extra fare and lasts an hour. Whether either is worth it depends on nothing about the journey and everything about **how often you make it**. Two trips a week and you should be buying singles.

Where it breaks: a season ticket is bought deliberately and shows up on a statement. A cache write is bought by a one-line marker, is invisible unless you read `usage`, and renews itself silently on every request that misses.

VISUAL ANALOGY

The post room

Batch processing is the post room. You hand over a sack of letters, they go out when the van goes, and you get everything back together. It is half the price of a courier and completely unsuitable for anything someone is waiting for. The 24-hour window is the van's timetable: it leaves when it leaves.

Where it breaks: the post room does not care what is in the envelopes, but the batch worker does. It runs the same agentic loop, calls server tools, and will hand you back an unfinished turn (`pause_turn`) that you must post again.

VISUAL ANALOGY

The bookmark that only works if someone put it there

The lookback window is a librarian who will check the last twenty pages before your bookmark for **another bookmark**, and will not read a word of the book to work out where you were. Stable content behind your bookmark is irrelevant; only previously placed bookmarks count.

Where it breaks: a librarian would eventually just read the book. The cache never does. It matches hashes, and twenty is a hard count of positions — not a heuristic, not a budget, and not affected by how large the blocks are.

30.15 Architecture: Where the Three Levers Sit

Batching, caching and cost control act at three different layers, and confusing them is how plans go wrong.

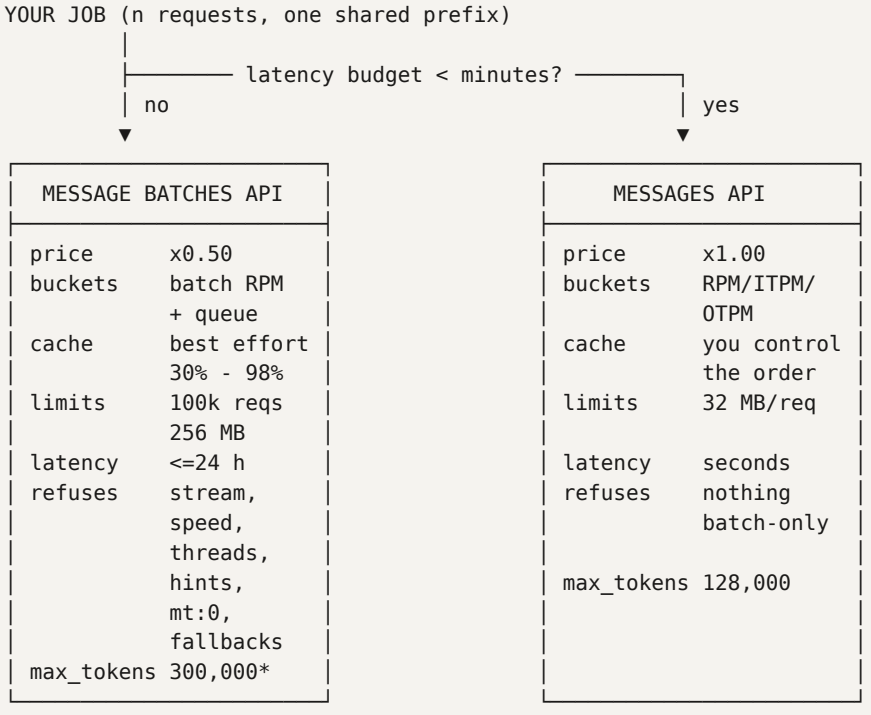
Caching acts inside a single request's prefill. It is invisible to your application logic and changes nothing about the answer — *"the response you receive is identical to what you would get if prompt caching were not used."* Its inputs are prompt **order** (Chapter 20: stable first, variable last) and prompt **stability** (Chapter 21: the system prompt as an authority boundary that happens to be a byte-stable string).

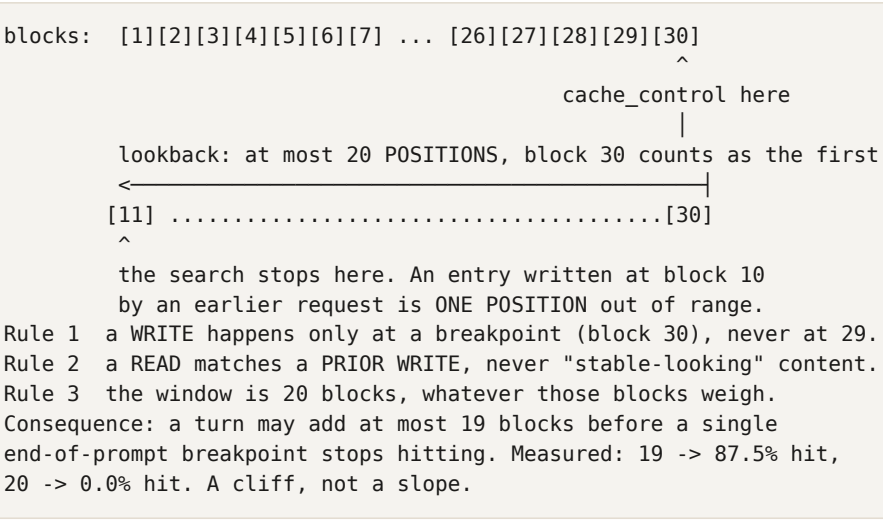
Batching acts at the scheduling layer, between your job runner and the API. It replaces a connection with a poll, which removes Chapter 29's entire timeout problem — there is no idle connection to abandon — and replaces it with an expiry problem instead. It also removes the job from the Messages rate-limit buckets entirely and puts it in its own.

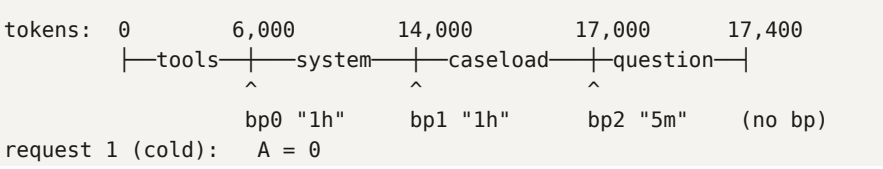
Cost controls act at the organisation layer, above everything, and are the only ones of the three that can actually stop work.

The routing decision belongs in one place, and it is a function of the request, not a setting: latency requirement, payload size, and expected cache hit rate on each path.

30.16 Figures







```
                      B = 14,000  (highest 1h breakpoint after A)
                      C = 17,000  (last breakpoint)
    billed:  read 0 | write_1h 14,000 | write_5m 3,000 | base 400
  request 2 (warm):   A = 17,000  (highest cache hit)
                      B = C = A   (no breakpoint past the hit)
    billed:  read 17,000 | write_1h 0 | write_5m 0 | base 400
  Constraint: a "1h" block must appear BEFORE any "5m" block, or 400.
```

30.17 Examples

Example 1 — the breakpoint in the wrong place. Costs exactly 1.25× not caching:

```
system = [
    {"type": "text", "text": TAXONOMY},              # 20,000 tokens
    {"type": "text", "text": f"Now: {utcnow()}",
     "cache_control": {"type": "ephemeral"}},        # changes every call
]
```

Example 2 — the same prompt, fixed. One marker moved:

```
system = [
    {"type": "text", "text": TAXONOMY,
     "cache_control": {"type": "ephemeral"}},        # last IDENTICAL block
    {"type": "text", "text": f"Now: {utcnow()}"},    # after the breakpoint
]
```

Example 3 — layered TTLs. Longer first, always:

```
tools = [*TOOLS[:-1], {**TOOLS[-1],
                       "cache_control": {"type": "ephemeral",
                                         "ttl": "1h"}}]
system = [{"type": "text", "text": POLICY,
           "cache_control": {"type": "ephemeral", "ttl": "1h"}},
          {"type": "text", "text": TODAYS_CASELOAD,
           "cache_control": {"type": "ephemeral"}}]     # 5m
```

Example 4 — reading the split, not the scalar.

```
u = response.usage
five = u.cache_creation.ephemeral_5m_input_tokens
hour = u.cache_creation.ephemeral_1h_input_tokens
assert u.cache_creation_input_tokens == five + hour
total_input = (u.cache_read_input_tokens
               + u.cache_creation_input_tokens + u.input_tokens)
```

Example 5 — a pre-warm that actually warms. Breakpoint on the shared block, effort held constant:

```
client.messages.create(
    model="claude-opus-5", max_tokens=0,
    output_config={"effort": "high"},          # same as real traffic
    system=[{"type": "text", "text": SYSTEM,
             "cache_control": {"type": "ephemeral"}}],
    messages=[{"role": "user", "content": "warmup"}])
```

Example 6 — matching batch results to requests. Order is not guaranteed:

```
for result in client.messages.batches.results(batch_id):
    record = by_custom_id[result.custom_id]      # never by position
    if result.result.type == "succeeded":
        record.verdict = read(result.result.message)   # Chapter 28
    elif result.result.type == "expired":
        record.requeue()                               # not billed
```

30.18 Code Walkthrough: The Algorithm, Not the Advice

`cacheplan.py` implements the documented cache algorithm rather than describing it, which is the only way to be sure the advice in §30.5 follows from the mechanism. The heart of it is twenty lines.

```
def prefix_hash(blocks, upto):
    """Cumulative hash of blocks[0..upto], inclusive. Order matters."""
    h = sha256()
    for b in blocks[:upto + 1]:
        h.update(b.digest().encode("utf-8"))
        h.update(b"\x1e")
    return h.hexdigest()
```

Cumulative and inclusive: that single property produces every cache rule in the chapter. Change block 3 and the hashes at 3, 4, 5 and everything after differ, which is Chapter 21's `tools` → `system` → `messages` hierarchy falling out of arithmetic rather than being asserted.

The read path is the part people get wrong, so it is written to be read:

```
    hit_index = -1
    for bp in reversed(breakpoints):
        for i in range(bp, max(-1, bp - LOOKBACK), -1):
            if cache.get(prefix_hash(blocks, i)) is not None:
                hit_index = i
                break
        if hit_index >= 0:
            break
```

`cache.get` is the whole argument. The loop asks *"is there an entry here?"*, never *"has this content changed?"*. There is no code path in which stable content behind a breakpoint becomes cached, because there is nowhere for such a path to live. `LOOKBACK` is 20 and the breakpoint counts as the first position, so the inner range covers exactly 20 indices.

Writes are one line, guarded by one condition:

```
    for bp, t in after:
        cache.put(prefix_hash(blocks, bp), cum[bp], t, now)
```

where `after` is the breakpoints strictly past the hit. Nothing writes at a non-breakpoint. Nothing writes behind a hit.

Two smaller decisions are deliberate. A hit **refreshes** the entry — *"the cache is refreshed for no additional cost each time the cached content is used"* — which turns the arrival model of §30.8 into a renewal

process and is why the closed form there is exact. And a prompt below the model's minimum returns a `Charge` with `cached=False` and everything at base rate, **without raising**, because the documentation is explicit that *"any requests to cache fewer than this number of tokens will be processed without caching, and no error is returned."* Modelling a silent failure as a silent failure is the point; `min_cacheable` raises `Unverified` only when the model is not in the verified table at all.

`batchpre.py` is the gate. It reproduces every documented batch rejection offline, and refuses to guess:

```
        if model in OUTPUT_300K_EXCLUDED:
            raise BatchRejected("%s is not on the %s model list"
                                % (model, OUTPUT_300K_BETA))
        if model not in OUTPUT_300K_MODELS:
            raise Unverified("%s support for %r is UNKNOWN (verified %s)"
                             % (OUTPUT_300K_BETA, model, VERIFIED_ON))
```

Two rungs, not one. Fable 5 and Haiku 4.5 are excluded **by omission from an exhaustive published list**, which is a documented negative. Mythos 5 appears on no list at all, which is not a documented negative — it is an absence of documentation, and Chapter 14's rule applies: **a fact is verified on a date or it is UNKNOWN, and UNKNOWN must raise rather than default.**

30.19 Mini Project: A Batch-versus-Online Decision Model, a Cache Planner and a Preflight

Folder structure. Six new files, four copied in and kept in sync.

```
ch30/
  cacheplan.py    the documented cache algorithm, priced
  arrivals.py     what a TTL buys, closed form and simulation
  batchpre.py     the batch and pre-warm preflight
  fixtures.py     recorded cases, every one marked DOC or BUILT
  decide.py       batch vs online: cost, buckets, latency, crossover
  report.py       every published figure, recomputed
  test_ch30.py    36 tests
  costmodel.py    copied from ch11/
  limits.py       copied from ch19/
  planner.py      copied from ch19/
  usage.py        copied from ch28/
```

Run `python3 report.py` for the figures and `python3 -m pytest` for the assertions. Nothing needs a key or a network.

Result 1 — 20 documented rejections offline, and four UNKNOWNs that raise

```
batch fixtures      accept=6  reject=15  UNKNOWN=1  wrong=0
pre-warm fixtures   accept=1  reject=5
total documented rejections reproduced: 20
facts recorded UNKNOWN (each raises): 4
  UNKNOWN  output-300k-2026-03-24 on Claude Mythos 5
  UNKNOWN  whether the automatic server-tool-result breakpoint consumes
           one of the four cache_control slots
  UNKNOWN  max_tokens: 0 pre-warming on Amazon Bedrock and Google Cloud
  UNKNOWN  whether a batched request may carry
```

```
            diagnostics.previous_message_id
```

Twenty rejections, none of which needed a key: the six unsupported parameters, server-side fallback, `max_tokens: 0` and sub-1 values, three `custom_id` violations, `output-300k` on two excluded models and without the header at all, and five pre-warm conflicts. Every fixture carries `DOC` or `BUILT`, and the three `BUILT` ones are constructed payloads exercising documented rules, not invented recordings.

Result 2 — the limit that binds is not the one anybody quotes

```
  bytes/token   requests/batch   bound by
          3.0             4364   size
          3.5             3741   size
          4.0             3273   size
          4.5             2909   size
400,000 requests at 4.0 bytes/token: 123 batches of 3273 (bound by size)
if the 100,000-request limit had bound: 4 batches
```

Bytes per token is a property of your payload rather than a published constant, so the table sweeps it — and the conclusion is the same at every value in the range. The nightly job needs **123 batches**, not four, because the size limit admits **30.6 times fewer** requests than the count limit would, and each of the 400,000 requests carries its own complete copy of the 20,000-token taxonomy. Chapter 16's statelessness has not gone anywhere; batching a shared-prefix workload is the most byte-redundant thing you can do with the API.

Result 3 — a cache configuration that costs 1.25× not caching

Ten requests, five 4,000-token static blocks and one 60-token block carrying a timestamp:

```
no caching at all                    units= 200600.0  itpm= 200600
breakpoint on the varying block      units= 250750.0  itpm= 200600
   writes=10 reads= 0  ratio to no caching = 1.2500
breakpoint at the end of the prefix  units=  43600.0  itpm=  20600
   writes= 1 reads= 9  ratio to no caching = 0.2173
```

Exactly 1.2500, reproducing Chapter 21's measurement from the algorithm rather than from a coincidence, and **ten writes with zero reads** as the diagnostic signature. Moving one marker one block earlier is a **5.75-fold** improvement in cost and, separately, a **9.74-fold** reduction in ITPM draw, because Chapter 19's rule holds: cache reads draw nothing from the input-token bucket. The failure is invisible in every place people look — no error, no warning, a healthy-looking `cache_creation_input_tokens` on every single request.

Result 4 — the lookback cliff

```
  added/turn   turns   hits   misses   hit rate
           5       8      7        1      87.5%
          10       8      7        1      87.5%
          15       8      7        1      87.5%
          19       8      7        1      87.5%
          20       8      0        8       0.0%
          25       8      0        8       0.0%
          30       8      0        8       0.0%
  25 added/turn with a second breakpoint at block 10: 7/8 hits
```

Nineteen added blocks per turn: every turn after the first hits. Twenty: **nothing hits, ever**. There is no intermediate regime and no warning at 18. One extra breakpoint, placed before the conversation grew, restores 7 of 8.

Result 5 — the break-evens, and a negative result

```
5m write=1.25x  break-even hit rate=21.7391%  break-even rate=2.9415/hour
1h write=2.00x  break-even hit rate=52.6316%  break-even rate=0.7472/hour
5m and 1h cost the same at 0.5477 requests/hour
  requests/hour   none      5m        1h        cheapest
         0.2500   1.0000    1.2263    1.5797    none
         0.5000   1.0000    1.2031    1.2524    none
         0.7472   1.0000    1.1806    1.0000    none
         1.0000   1.0000    1.1581    0.7990    1h
         2.0000   1.0000    1.0735    0.3571    1h
         2.9415   1.0000    1.0000    0.2003    1h
         6.0000   1.0000    0.7975    0.1047    1h
        60.0000   1.0000    0.1077    0.1000    1h
```

Read the "cheapest" column downwards. **The 5-minute TTL never appears in it.** Below 0.7472 requests per hour on a prefix the right answer is no caching at all; above it the 1-hour cache is cheaper at every rate tested, converging on the 1-hour cache's floor of 0.1000 against the 5-minute cache's 0.1077 at sixty requests an hour. The test suite asserts the general claim across eleven rates from 0.1 to 600, and asserts it as a **weak** inequality, because above a few hundred requests an hour the two TTLs tie to floating-point resolution rather than one beating the other.

This contradicts documented guidance — *"If you have prompts that are used at a regular cadence… continue to use the 5-minute cache, because this will continue to be refreshed at no additional charge"* — so it is published as a negative result with its assumption named. The closed form was checked against a seeded 200,000-draw simulation and agreed to within **0.0009** at every rate and TTL tested. The assumption doing the work is **memorylessness**: an exponential gap distribution has no notion of a session.

Result 5b — the distribution that overturns it

Eight requests thirty seconds apart, then a long silence, five hundred times over:

```
one session every 4 hours     5m hit=0.8750 cost=0.24375 |
                              1h hit=0.8750 cost=0.33750
one session every 45 minutes  5m hit=0.8750 cost=0.24375 |
                              1h hit=0.9998 cost=0.10047
at a 4-hour inter-session gap the 5m cache is 27.78% cheaper than the
1h cache
```

When the inter-session gap exceeds an hour, **both TTLs achieve the identical 87.5 per cent hit rate** — the seven within-session requests hit, the first of each session misses — and the 1-hour cache simply pays twice as much for every one of those misses. It is **27.78 per cent** more expensive for exactly nothing. Bring the sessions to 45 minutes apart and the 1-hour cache wins by 2.43×.

So the honest statement is not "use the 1-hour cache". It is: **the TTL is a bet on the gap distribution, and you must know whether your traffic has sessions.** Two further considerations the cost model deliberately omits, both documented: the 1-hour cache also buys **rate-limit headroom**, because cache

reads draw nothing from ITPM, and it buys **time-to-first-token** across a gap. Either can justify a TTL the dollar arithmetic rejects — but you should know you are buying them.

Result 6 — mixed TTLs bill at A, B and C

```
request 1  read=0      write_1h=14000  write_5m=3000  base=400
request 2  read=17000  write_1h=0      write_5m=0     base=400
out-of-order TTLs rejected: 400 invalid_request_error: a 1h
  cache_control block must appear before any 5m block
below the 512-token minimum on Opus 5: cached=False, base=400,
  no error raised
undocumented model: minimum cacheable length for 'claude-opus-9' is
  UNKNOWN
```

The three positions behave exactly as documented, the ordering constraint is enforced locally, the sub-minimum prompt is **silently** uncached, and an unverified model raises.

Result 7 — the decision, and where the discount stops paying

```
  online, cache hit 0.98   $  13920.00
  online, cache hit 0.95   $  15300.00
  online, no caching       $  49000.00
  batch, cache hit 0.98    $   6960.00
  batch, cache hit 0.65    $  14550.00
  batch, cache hit 0.30    $  22600.00
  batch, no caching        $  24500.00
batch at the bottom of the documented 30-98% band costs 62.4% MORE
  than the online path at 0.98
crossover batch hit rate vs online 0.98: 67.7391%
crossover batch hit rate vs online 0.95: 61.7391%
```

The crossover sits **inside** the documented 30-to-98 per cent band, which is the whole finding: the batch path is not reliably cheaper than a well-cached online path, and the price list cannot tell you which side you are on. Note also that batching an **uncached** workload halves it perfectly — $49,000 to $24,500 — so the discount is real; it is the interaction with a cache you already had that inverts the ranking.

Result 8 — the buckets, and why "just run it online" is not a plan

```
online in 60 min, Build tier: rpm=6667 (133.3%) itpm=6000000 (120.0%)
  otpm=5333333 (533.3%) feasible=False
    60 min: rpm 133.3%  itpm 120.0%  otpm 533.3%  -> over limit
   180 min: rpm  44.4%  itpm  40.0%  otpm 177.8%  -> over limit
   360 min: rpm  22.2%  itpm  20.0%  otpm  88.9%  -> feasible
   480 min: rpm  16.7%  itpm  15.0%  otpm  66.7%  -> feasible
batch, Build tier: 123 batches, queue 400000/300000 = 133.3% of the
  documented queue limit, Messages-bucket draw 0
```

Three findings in one table. The binding bucket online is **OTPM at 533 per cent**, not the requests-per-minute figure people quote; the online path cannot drain this job in under **six hours** on Build no matter how well the cache performs, because output tokens are not cacheable. The batch path draws **exactly nothing** from the Messages buckets — Chapter 19's separate RPM and queue — so the two paths do not compete, which is an argument for splitting a workload rather than choosing between paths. And the batch **queue** limit binds at 133.3 per cent on Build: 400,000 requests waiting against a

documented 300,000, so the job must be released in waves or the tier must move.

What the project establishes

Four numbers are worth carrying out of it. A cache break-even of **21.74 per cent** (5m) and **52.63 per cent** (1h) hit rate. A lookback cliff at **20 blocks**. A batch/online crossover at a **67.74 per cent** batch hit rate. And one ratio: **1.2500**, the price of a breakpoint in the wrong place.

30.20 Hands-on Labs

Lab 1 — Prove the 1.25× (success: you reproduce `1.2500` exactly). Build a ten-request workload with five static blocks and one varying block. Put `cache_control` on the varying block, price it, and divide by the uncached cost.

Lab 2 — Find your own lookback cliff (success: two adjacent values of *k* with hit rates 87.5% and 0.0%). Sweep the number of blocks added per turn from 15 to 25 with a single end-of-prompt breakpoint.

Lab 3 — Price your real TTL (success: a defended choice of `5m`, `1h` or none). Take one week of timestamps for requests sharing a prefix, compute the empirical gap distribution, and evaluate all three policies with `arrivals.price_gaps`. State whether your traffic has sessions.

Lab 4 — Find the batch crossover for your workload (success: a hit rate between 0 and 1, or a proof that batch always wins). Use `decide.crossover_batch_hit` with your own prefix, variable and output token counts, and your measured online hit rate.

Lab 5 — Make the preflight fail (success: 20 rejections and at least one `Unverified`). Feed `check_batch` a list containing a duplicate `custom_id`, a `stream: true`, and a 300,000-token request on Claude Mythos 5.

30.21 Lab Solutions

Lab 1. The ratio is `1.25` exactly and independent of the token counts, which is the point — it is the write multiplier with no read ever taken, so no arithmetic other than the multiplier survives:

```
cache, units = C.Cache(), 0.0
for r in range(10):
    units += C.run_request(prefix_blocks(r), [5], cache,
                           now=r * 60).billed_units()
assert cache.writes == 10 and cache.reads == 0
assert units == 1.25 * 10 * 20_060
```

Generalise it: with hit rate `h`, cost is `(1 − h)·w + h·0.1`, so a breakpoint that guarantees `h = 0` always costs exactly `w`.

Lab 2. The cliff is between 19 and 20. The condition is `N_t − N_{t−1} ≤ 19`, because the window covers `N_t − 1` down to `N_t − 20` and the previous write sits at `N_{t−1} − 1`. Two breakpoints, one static and one at the end, guarantee at least the static hit forever.

Lab 3. Compute `hit_rate_from_gaps(gaps, ttl)` for both TTLs, then `cost_multiplier`. If both TTLs give the **same** hit rate, your traffic has sessions longer than the gap between them, and the shorter TTL wins automatically — that is Result 5b. If the 1-hour hit rate is materially higher, the 1-hour cache almost certainly wins.

Lab 4. Setting `0.5·(in(h_b) + out) = in(h_o) + out` in base-input units gives a linear equation in `h_b`, implemented in `crossover_batch_hit`. If it returns a value above 1.0, batch always wins for you — which happens whenever output dominates input, because output is discounted 50 per cent and is never cacheable.

Lab 5. Order matters in the failure you get: `check_batch` validates `custom_id` before `params`, so a duplicate id raises before a `stream: true` in the same list is ever seen. Real batch validation is asynchronous and reports at the end, which is exactly why a local preflight that fails fast is worth having.

30.22 Best Practices

BEST PRACTICE

1. **Put the breakpoint on the last identical block**, and prove it by asserting `cache_read_input_tokens > 0` on the second request of a test.
2. **Price the cache before enabling it.** Below 21.74 per cent (5m) or 52.63 per cent (1h) expected hit rate, the correct configuration is no `cache_control`.
3. **Choose the TTL from the gap distribution, not the prompt.** Ask whether your traffic has sessions, and how long the silences are.
4. **Add a second breakpoint before you need it.** A write must already exist within 20 blocks; adding the breakpoint after the miss does not retroactively create one.
5. **Log all three usage fields** — `input_tokens`, `cache_creation_input_tokens`, `cache_read_input_tokens` — and the `cache_creation` split when two TTLs are in play.
6. **Alert on writes without reads.** A sustained ratio near 1.0 means the breakpoint is on a moving block.
7. **Always match batch results by** `custom_id`, never by position.
8. **Preflight batches locally**, because the API validates asynchronously and may report 24 hours later.
9. **Size batches by bytes first**, then by count.
10. **Treat** `expired` **as free but late**, and re-queue with the same `custom_id` so the job is idempotent at the record level — Chapter 29 showed the protocol will not do this for you.
11. **Hold the model,** `tool_choice`**, thinking configuration and** `effort` **constant** for the life of a cached conversation.
12. **Use** `defer_loading` **and tool search** to grow a toolset without touching the `tools` prefix.
13. **Pre-warm with the same configuration as real traffic**, or you write an entry nothing will read.
14. **Set workspace spend caps knowing a batch may slightly exceed them**, and reconcile against the Usage and Cost API rather than against your own counters.

30.23 Common Mistakes

COMMON MISTAKE

The breakpoint on the timestamp

Symptom: `cache_creation_input_tokens` is large on every request; `cache_read_input_tokens` is always 0; the bill is 25 per cent above the uncached estimate. **Cause:** `cache_control` sits on a block containing a timestamp, request ID or the user's message. The hash differs every time, and the lookback finds no earlier write because writes happen only at breakpoints. **Fix:** move the marker to the last block that is byte-identical across requests. **Detection:** assert `cache_read_input_tokens > 0` on the second request of an integration test; or send `cache-diagnosis-2026-04-07` and read `system_changed`.

COMMON MISTAKE

Assuming the cache finds stable content

Symptom: a large stable system prompt is never cached, despite an explicit breakpoint further down the prompt. **Cause:** believing the lookback searches for stability. It searches for **prior writes**. **Fix:** put a breakpoint at the end of the stable region so a write exists there. **Detection:** count breakpoints in your request builder; every region you expect to be cached needs one.

COMMON MISTAKE

The agent loop that outruns the window

Symptom: cache hits work for two turns then stop permanently, in a conversation that only ever appends. **Cause:** the turn adds 20 or more blocks — commonly a fan-out of parallel `tool_result` blocks — so the previous write falls outside the 20-position window. **Fix:** a second breakpoint nearer the growing end, added before the growth. **Detection:** log blocks-per-turn; anything at or above 20 is a defect.

COMMON MISTAKE

Buying the 1-hour TTL for bursty traffic

Symptom: costs rise after switching to `"ttl": "1h"`, and the hit rate does not move. **Cause:** the traffic is sessions separated by gaps longer than an hour. Both TTLs miss on the first request of each session; the 1-hour cache pays 2× for those misses instead of 1.25×. **Fix:** revert to 5 minutes, or pre-warm at the start of a session. **Detection:** compare hit rates under both TTLs on the same week of timestamps; identical hit rates mean the longer TTL is buying nothing.

COMMON MISTAKE

Batching for the discount alone

Symptom: a job moves to Batch and the bill goes up. **Cause:** the online path had a near-perfect cache and the batch path gets best-effort hits somewhere in the documented 30-to-98 per cent band. **Fix:** compute the crossover; ensure identical `cache_control` blocks in every request in the batch, keep a steady stream of batches, and consider the 1-hour TTL, which the documentation recommends precisely because batches take longer than five minutes. **Detection:** compare `cache_read_input_tokens` summed across batch results against the same figure online.

COMMON MISTAKE

Sizing a batch by request count

Symptom: a 413 `request_too_large` on a batch of 20,000 requests, well under the documented 100,000. **Cause:** every request carries its own copy of a large shared prefix; 256 MB is reached first. **Fix:** size by serialised bytes and chunk accordingly. **Detection:** measure `len(json.dumps(batch).encode())` before submitting.

COMMON MISTAKE

Caching below the model's minimum

Symptom: both cache usage fields are 0 and no error was returned. **Cause:** the cached prefix is shorter than the model's minimum — 512 tokens on Opus 5 but **4,096 on Haiku 4.5**. **Fix:** extend the cached region past the threshold, or accept that this prompt is not cacheable on this model. **Detection:** a preflight that knows the per-model minimum and refuses to guess for unlisted models.

COMMON MISTAKE

Reading `cache_creation_input_tokens` as the whole story

Symptom: a cost model under-reports when two TTLs are in use, or an ITPM plan is wrong. **Cause:** the scalar is the sum of the `cache_creation` object; the 5-minute and 1-hour halves are priced differently, and server tools can add 5-minute writes you did not request. **Fix:** read `cache_creation.ephemeral_5m_input_tokens` and `ephemeral_1h_input_tokens` separately. **Detection:** assert the scalar equals the sum, and alert when 5-minute writes appear in a 1-hour-only request.

30.24 Performance Considerations

Caching improves **prefill**, which Chapter 11 measured as the smaller of the two latency terms for most workloads and the dominant one for long-context prompts. It does nothing for decode, so a request generating 800 tokens is not meaningfully faster for having cached 20,000 input tokens — the 800 tokens are still generated one at a time. Both TTLs *"behave the same with respect to latency."*

Two constraints shape concurrency. *"A cache entry only becomes available after the first response begins"*, so firing 500 parallel requests against a cold prefix produces 500 cache writes and no reads. Warm the prefix with one request — or a `max_tokens: 0` pre-warm — and then fan out. And on the batch path, concurrency is exactly why hits are best-effort: many workers start against the same cold prefix at once.

On throughput, the decisive figure from Result 8 is that **OTPM, not RPM, binds a bulk job**, at 533 per cent against 133 per cent for a one-hour drain. Output tokens cannot be cached, discounted only by Batch's 50 per cent, and bounded only by generating fewer of them — Chapter 11's running thread that generating fewer tokens is the one lever improving latency, throughput and cost together.

Batch latency has no percentile you can plan against. *"Most batches complete within 1 hour"* is a description, not a service level; the only contractual number is 24 hours. Design so that a 24-hour result is still useful, and so that `expired` records are re-queued rather than lost.

30.25 Security Considerations

Cache isolation is a workspace boundary. Caches are isolated between organisations always, and between workspaces on the Claude API, Claude Platform on AWS and Microsoft Foundry — but **per organisation only on Amazon Bedrock and Google Cloud**. A tenancy design that relies on workspace isolation for cache separation does not port to those two platforms unchanged.

Prompt caching is ZDR eligible. *"KV cache representations and cryptographic hashes of cached content are held in memory only and are not stored at rest"*, with a minimum lifetime of the TTL and prompt deletion afterwards. Cache diagnostics is ZDR eligible (qualified), and its fingerprints are *"only cryptographic hashes and token-count estimates"*.

Batch storage is not ephemeral. Batch processing *"stores request and response data for up to 29 days after batch creation"*, because asynchronous processing requires it. If a workload cannot tolerate 29 days of server-side retention, use `DELETE /v1/messages/batches/{batch_id}` after retrieval — and cancel first if the batch is still running. This is the sharpest operational difference between the two paths and belongs in a data-protection assessment, not in a performance review.

`custom_id` **is data you chose.** It appears in results and Console views. Do not put customer identifiers, email addresses or case numbers in it; use an opaque key and hold the mapping yourself — the same reasoning Chapter 27 recorded for `metadata.user_id`, which *"should be a uuid, hash value, or other opaque identifier"*.

A cached prefix is a shared authority surface. Chapter 21 established that the system prompt is an authority boundary. Caching does not change who may write there, but it does mean a poisoned prefix persists for up to an hour and is read by every subsequent request that matches it, at a tenth of the price. Validate what goes into the cached region with the same care you would apply to the system prompt itself; Chapter 58 takes prompt injection up properly.

30.26 Real Production Examples

A nightly classification job. The chapter's running example. Cache the taxonomy behind a 1-hour breakpoint, batch the tickets, size the batches by bytes, match by `custom_id`, re-queue `expired`. Expect 123 batches and a queue that exceeds the Build tier's limit, so release in waves.

A support assistant with a 40,000-token knowledge base. Sessions of six to ten turns, then silence for hours. Result 5b applies directly: both TTLs achieve the same hit rate, so the **5-minute** cache is correct, with a pre-warm fired when the session opens if time-to-first-token matters.

A coding agent. Twelve parallel tool calls per turn is normal, so the 20-block window is the binding constraint and a second breakpoint is mandatory. Use `defer_loading` so discovered tools arrive as `tool_reference` blocks in the history rather than as edits to `tools`, which would invalidate everything.

An evaluation harness. Chapter 24's regression suite is the canonical batch workload: hundreds of pinned cases, no latency requirement, one shared prompt. Batch it, cache the shared prefix with a 1-hour TTL, and note that Chapter 24 costed a properly powered suite at **$2,007.72 a month** — halving that is the single easiest saving in this book.

A multi-tenant platform. Per-tenant workspaces give clean billing attribution and clean cache isolation, and cost you cache sharing on any prefix common to tenants. Decide deliberately, and measure the hit rate on both sides of the decision before committing.

30.27 Certification Notes

Concept	Exam domain	Weight
Batch limits, expiry, retention, `custom_id`	D5 Context Management & Reliability	15%
What Batch rejects and why	D5 Context Management & Reliability	15%
Prompt caching mechanism, breakpoints, lookback	D5 Context Management & Reliability	15%
Cache pricing, TTL selection, break-even	D1 Agentic Architecture & Orchestration	27%
Caching in an agent loop, tool prefix stability	D2 Tool Design & MCP Integration	18%
Prompt ordering for cacheability	D4 Prompt Engineering & Structured Output	20%
Spend caps, Usage and Cost API, `inference_geo`	D1 Agentic Architecture & Orchestration	27%

CERTIFICATION NOTE

Exam Tips

- **50 per cent off is not 50 per cent cheaper.** If a question gives you a cache hit rate on each path, compute; the discount is on tokens, not on the bill.
- **The lookback window is 20 blocks, not 20 tokens and not 20 turns.**
- A prompt below the minimum is **not cached and returns no error**. Distractors that promise a 400 are wrong.
- Minimums are **not** ordered by model recency: Opus 5 is 512, Haiku 4.5 is 4,096.
- `input_tokens` means **tokens after the last breakpoint**, not total input. Total is the sum of all three fields.
- Batch rejects `stream`, `speed`, Threads fields, routing hints, `max_tokens: 0` and research preview — everything that tunes a connection or a schedule.
- Results are **unordered**; `custom_id` is the only join key.
- Retention is **29 days from** `created_at`, expiry is **24 hours**, and `expired`, `errored` and `canceled` are **not billed**.
- 1-hour writes are **2×**, 5-minute writes **1.25×**, reads **0.1×**, and reads **do not count against ITPM**.
- Anything documented neither way — `output-300k` on Mythos 5, for instance — is **UNKNOWN**, and the correct exam answer is the one that refuses to assume.

30.28 Twenty Practice Questions

1. A batch of 40,000 requests, each carrying a 25,000-token shared prefix, returns 413 `request_too_large`. Why?

2. `cache_creation_input_tokens` is 18,000 on every request and `cache_read_input_tokens` is always 0. What is wrong?
3. Both cache usage fields are 0 and no error was returned. Give the two most likely causes.
4. What is the break-even hit rate for the 1-hour cache, and why?
5. A conversation appends 22 blocks per turn with one breakpoint at the end. What happens at turn 3, and what fixes it?
6. Which stop reason does a `max_tokens: 0` pre-warm return, and what is in `content`?
7. Name three parameters the Batches API rejects and give the reason common to all three.
8. Batch results arrive in a different order from the requests. How do you match them?
9. What does `input_tokens` mean in a response that used caching?
10. On which platforms is automatic caching unavailable?
11. Which is invalidated by toggling web search: tools, system, or messages?
12. Your team switches from `5m` to `1h` and the hit rate does not change but the bill rises. Explain.
13. What is the maximum `max_tokens` on the Batches API, under what header, and on which models?
14. Why can a batch slightly exceed a workspace spend limit?
15. Which usage fields count against ITPM?
16. What does `cache_miss_reason: {"type": "previous_message_not_found"}` tell you about your prompt?
17. A batch result has `stop_reason: "pause_turn"`. Is the turn finished?
18. Where must a `"ttl": "1h"` block appear relative to a `"5m"` block, and what happens otherwise?
19. Give the three billing positions when TTLs are mixed and what each is charged at.
20. Batch is 50 per cent off. Under what circumstances is it more expensive than the online path?

30.29 Answers

1. The 256 MB limit binds before the 100,000-request limit. Every request carries its own copy of the prefix; at ~4 bytes per token that is ~100 kB each, so roughly 2,600 requests fill the batch. **Wrong distractors:** the request count (40,000 < 100,000); the 32 MB Messages limit (that is per synchronous request); a rate limit (413 is a size error, not a 429).
2. The breakpoint sits on a block that changes every request. Every request writes and none reads, costing exactly 1.25× not caching. **Wrong:** "the cache is disabled" — writes are happening; "the prompt is too short" — that gives zeros in **both** fields.
3. Either the cached prefix is below the model's minimum, or no `cache_control` was sent at all. The first is silent by design.
4. **52.63 per cent**. Cost is `(1 − h)·2.0 + h·0.1` against 1.0; solving gives `(2 − 1)/(2 − 0.1)`.
5. Turn 3's lookback covers 20 positions and the turn-2 write is outside it, so there is no hit and the whole prompt is re-prefilled. A second breakpoint placed nearer the growing end **from the start** fixes it. **Wrong:** raising the TTL — the entry has not expired, it is out of range.
6. `stop_reason: "max_tokens"` with `content: []` and a fully populated `usage`. Zero output tokens are billed.

7. `stream: true`, `speed`, `store`/`previous_thread_event_id`, `cache_hint`/`context_hint`, `max_tokens: 0`, `research_preview_2026_02`. Each tunes a connection or a schedule, and a batch has neither.
8. By `custom_id`, which is unique within the batch and matches `^[a-zA-Z0-9_-]{1,64}$`. Position is explicitly not guaranteed.
9. Only the tokens **after the last cache breakpoint**. Total input is `cache_read_input_tokens + cache_creation_input_tokens + input_tokens`.
10. **Amazon Bedrock.** It is available on the Claude API, Claude Platform on AWS, Google Cloud and Microsoft Foundry.
11. **System and messages.** The tools cache survives, because the toggle modifies the system prompt.
12. The traffic is bursty sessions with gaps longer than an hour, so both TTLs miss on the same requests — the first of each session — and the 1-hour cache pays 2× rather than 1.25× for each miss. Measured at **27.78 per cent** more expensive.
13. **300,000**, with `output-300k-2026-03-24`, on Opus 5, Opus 4.8, Opus 4.7, Opus 4.6, Sonnet 5 and Sonnet 4.6. Not on Fable 5 or Haiku 4.5; Mythos 5 is **UNKNOWN**.
14. High throughput and concurrent processing mean many requests are in flight when the cap is evaluated. The documentation states this explicitly.
15. `input_tokens` and `cache_creation_input_tokens`. `cache_read_input_tokens` **does not** — the one documented exception being Claude Haiku 3.5.
16. **Nothing.** It means no fingerprint was stored for that id — usually a missing beta header on the previous turn, a different workspace, or too long a gap. It is *"not evidence that your request changed."*
17. **No.** The batch worker runs more server-loop iterations than a synchronous request before pausing, but a `pause_turn` result is still an unfinished turn; continue it by submitting the paused assistant content in a follow-up request.
18. The 1-hour block must appear **before** any 5-minute block. Otherwise the request is rejected.
19. **A** = tokens at the highest cache hit, charged as a **read** (0.1×); **B − A** = up to the highest 1-hour breakpoint after A, charged as a **1-hour write** (2×); **C − B** = up to the last breakpoint, charged as a **5-minute write** (1.25×). Everything after C is ordinary input.
20. When the online path has a materially better cache hit rate. With a 20,000-token prefix and 98 per cent online hits, batch loses below a **67.74 per cent** batch hit rate — inside the documented 30-to-98 per cent range.

30.30 Ten Practical Exercises

1. Instrument one production endpoint to log all three usage fields and plot the write-to-read ratio for a week.
2. Compute your empirical gap distribution per prefix and evaluate all three TTL policies against it.
3. Add an integration test that asserts `cache_read_input_tokens > 0` on the second identical request.
4. Count blocks added per turn in your agent loop and alert at 18.
5. Measure serialised bytes per request and compute how many fit in 256 MB.
6. Move your regression suite (Chapter 24) to Batch and measure the actual saving, including hit rates.

7. Enable `cache-diagnosis-2026-04-07` in staging and record which `cache_miss_reason` types you see.
8. Build a pre-warm job and measure time-to-first-token with and without it.
9. Reconcile one month of the Usage and Cost API against your own request counter and explain every discrepancy.
10. Audit every place your code interpolates a value into `system` or `tools`.

30.31 Five Challenge Labs

1. **The four-slot optimiser.** Given a prompt as a list of blocks with per-block change frequencies, choose at most four breakpoints and two TTLs minimising expected cost. Prove your solution beats a single end-of-prefix breakpoint on at least one realistic input.
2. **The batch scheduler.** Given 400,000 requests, a queue limit and a 24-hour window, emit a release schedule that never exceeds the queue and maximises cache hits by keeping a steady stream in flight.
3. **The hit-rate estimator.** From a week of production traffic, predict next week's cache hit rate under each TTL and validate against what actually happened.
4. **The invalidation fuzzer.** Mutate one field at a time — `tool_choice`, an image, `effort`, a tool description — and confirm each invalidates exactly the levels §30.10's table says it does.
5. **The honest dashboard.** Build a view showing, per prefix: hit rate, write-to-read ratio, dollars spent on writes that were never read, and the counterfactual cost of no caching at all.

30.32 Ten Interview Questions

1. Explain prompt caching to someone who has never used it, in three sentences, without saying "it caches the prompt".
2. Why does the lookback search for prior writes rather than for stable content?
3. When is caching more expensive than not caching, and what is the exact threshold?
4. How would you choose a TTL, and what data would you need?
5. Why is a 50 per cent discount not a 50 per cent saving?
6. What is the difference between a spend cap and a rate limit?
7. Which bucket usually binds a bulk job, and why can caching not help it?
8. What does a client-side timeout have to do with batching?
9. How does workspace design affect cache hit rate?
10. Your bill went up 25 per cent the day you enabled caching. Walk me through the diagnosis.

30.33 Summary

Part 5 began with a front door and ends with a bill. Chapters 26 to 29 established what a request is, what its parameters mean, what comes back and what to do when nothing does. Chapter 30 asks the question those four defer: what does all of it cost, and which of the levers marketed as savings actually save anything for your workload?

The answer is that all three levers are conditional, and each has a regime in which it costs money. Prompt caching is a cumulative prefix hash with writes only at breakpoints and a twenty-position backward search for previous writes. Everything follows from that sentence: a breakpoint on a block that changes costs exactly 1.25 times not caching, because every request writes and none reads; stable content behind a breakpoint is never found, because the search looks for writes and not for stability; and an agent turn adding twenty or more blocks falls off a cliff rather than down a slope, from an 87.5 per cent hit rate to zero with no intermediate state. The break-even is one line of algebra — `h* = (w − 1)/(w − 0.1)`, giving 21.74 per cent for the five-minute cache and 52.63 per cent for the one-hour — and below it the correct configuration is no `cache_control` at all, a configuration the API will never suggest and never warn you about.

The TTL question turned out to be a question about arrival distributions rather than about prompts. Under a memoryless model the five-minute default is strictly optimal at no arrival rate whatsoever: below three-quarters of a request an hour on a prefix nothing should be cached, and above it the one-hour cache is cheaper everywhere. That result is published as a negative one because it disagrees with documented guidance, and it is published together with the distribution that overturns it — bursty sessions separated by silences longer than an hour, where both TTLs achieve the identical 87.5 per cent hit rate and the one-hour cache is 27.78 per cent more expensive for nothing at all. Which is to say: the finding was never about which TTL is better. It was about the fact that a cache configuration is a bet on a distribution, and most teams have never looked at theirs.

Batching is the cleanest of the three levers and still not simple. It halves the price of every token and removes the job from the Messages rate-limit buckets entirely, which is worth as much as the discount when the binding constraint is output tokens per minute at five hundred and thirty-three per cent of a tier limit. But it competes with the cache rather than adding to it: at the bottom of the documented thirty-to-ninety-eight per cent best-effort band, the batch path costs sixty-two per cent more than a well-cached online path, and the crossover at 67.74 per cent sits inside the band, so the price list cannot tell you which side you are on. Meanwhile the limit that actually binds a batch is almost never the one people quote — a shared prefix repeated in four hundred thousand requests fills 256 MB after three thousand two hundred and seventy-three of them, so the job needs a hundred and twenty-three batches rather than four.

What ties all of it together is the unit. A token has three prices — a seat in the context window, a draw on a rate-limit bucket and a line on the invoice — and this chapter is the first place where all three move independently. A cache read is free on the bucket, a tenth on the invoice and full price on the seat. A batch is half price on the invoice, absent from the Messages buckets and unchanged on the seat. An output token is none of the above: it cannot be cached, it draws on its own bucket, and the only way to reduce it is to generate fewer. Part 6 opens with tool use, where the prompt starts changing shape between turns and every one of these rules is put under pressure.

KEY TAKEAWAYS

1. **A write happens only at a breakpoint, and a read looks for prior writes** — never for stable-looking content.
2. **The lookback window is 20 blocks**, counted as positions and not tokens; a turn may add at most 19 before a single end-of-prompt breakpoint stops hitting.
3. `h* = (w − 1)/(w − 0.1)`: caching pays above a **21.74 per cent** hit rate at 5 minutes and **52.63 per cent** at 1 hour, and costs money below.
4. **A breakpoint on a varying block costs exactly 1.25× not caching**, with ten writes and zero reads as the signature.
5. **The TTL is a bet on the gap distribution.** Identical hit rates under both TTLs mean the longer one is buying nothing — measured at 27.78 per cent more expensive for the same result.
6. **Below the model's minimum nothing is cached and no error is returned**, and the minimum is not ordered by recency: 512 on Opus 5, 4,096 on Haiku 4.5.
7. **Batch halves the invoice and vacates the Messages buckets**, but competes with your cache: 62.4 per cent more expensive at a 30 per cent batch hit rate, with the crossover at **67.74 per cent**.
8. **Size a batch by bytes.** 256 MB holds 3,273 requests of a 20,500-token prompt, not 100,000.
9. `input_tokens` **is what comes after the last breakpoint.** Total input is the sum of all three usage fields, and only reads escape ITPM.
10. **Every one of these levers is silent when misconfigured.** The only defence is to log all three usage fields and assert on them.

30.34 Glossary

Term	Definition
Automatic caching	A top-level `cache_control` field; the API places the breakpoint on the last cacheable block and moves it forward
Base-input unit	One token at the model's standard input price; every input token costs 0.1, 1.0, 1.25 or 2.0 of these
Batch queue limit	The number of batched requests that may wait for processing: 200,000 / 300,000 / 500,000 by tier
`cache_control`	The marker creating a cache breakpoint; `{"type": "ephemeral"}` with optional `"ttl": "1h"`
`cache_creation`	The usage object splitting writes into `ephemeral_5m_input_tokens` and `ephemeral_1h_input_tokens`
Cache diagnostics	Beta `cache-diagnosis-2026-04-07`; reports the first divergence between consecutive requests
`cache_miss_reason`	A discriminated union of six types naming why two requests diverged
Cache breakpoint	The position where a write occurs; at most four per request

Term	Definition
`custom_id`	Per-request identifier in a batch, `^[a-zA-Z0-9_-]{1,64}$`, the only correct join key for results
Extended output	`output-300k-2026-03-24`, raising `max_tokens` to 300,000 on the Batches API only
Hit rate	The fraction of requests reading a cached prefix rather than writing one
Lookback window	The 20 positions searched backwards from a breakpoint for a prior write
Pre-warming	A `max_tokens: 0` request that writes the cache and returns empty content
`processing_status`	A batch's lifecycle field: `in_progress`, `canceling`, `ended`
`request_counts`	Five counters on a batch: processing, succeeded, errored, canceled, expired
Spend cap	An organisation-level monthly limit that pauses usage rather than throttling it
TTL	Cache entry lifetime, 5 minutes or 1 hour, priced at 1.25× and 2× base input to write
Write multiplier	The premium paid to create a cache entry: 1.25× at 5 minutes, 2× at 1 hour

30.35 Further Reading

Primary sources, all verified 2 August 2026 at `platform.claude.com/docs/en`:

- `build-with-claude/batch-processing` — limits, result types, pricing, extended output, the server-tool loop.
- `build-with-claude/prompt-caching` — the three principles, the 20-block window, minimums, invalidation, mixed TTLs, pre-warming.
- `build-with-claude/cache-diagnostics` — the beta header, the six miss reasons, and the diagnostics-against-usage matrix.
- `agents-and-tools/tool-use/tool-use-with-prompt-caching` — breakpoints on tools, `defer_loading`, and the automatic server-tool-result breakpoint.
- `about-claude/pricing` and `api/rate-limits` — the multipliers, the tiers and the batch buckets.

Note that the `platform.claude.com/docs/en/api/...` reference pages are client-rendered and return nothing useful to a plain HTTP fetch; the prose guides above are the retrievable source, and this chapter says so rather than paraphrasing from memory.

Where to go next in this book. Chapter 31 opens Part 6 with how tool use works, where the prompt starts changing between turns and §30.6's window becomes a design constraint. **Chapter 35** returns to caching inside a tool-chaining loop. **Chapter 54** covers compaction and summarisation, which rewrite history and therefore invalidate everything after the rewrite. **Chapter 57** builds the observability that makes a write-to-read ratio visible before it becomes a bill. **Chapter 60** takes scaling, caching and cost optimisation up as a production discipline, with this chapter's break-evens as its inputs.